

Jour 8 : API Socket & Data Injection

1. Concepts & Définitions

- **Socket (Prise)** : Une abstraction logicielle qui sert de point de terminaison pour envoyer ou recevoir des données.

Pourquoi ? Pour qu'une application (exemple: Navigateur) n'ait pas à gérer les entêtes IP/TCP elle-même.

- **User Space vs Kernel Space** : La séparation entre le code privilégié (OS/Stack) et le code utilisateur (Appli).

Pourquoi ? Si l'appli plante, elle ne doit pas faire planter tout le système.

On a déjà vu cela dans le projet précédent au dernier jour

3. Test & Interprétation

1. Depuis un autre terminal on lance : **echo "un message " | nc 10.0.0.2 80.**

car on a configuré l'application à écouter sur le port 80, si vous mettez un autre port alors bien sur la connexion va s'établir mais le message sera envoyé au kernel space donc vous ne verrez pas l'affichage du message comme en dessous :

Résultats :

```
root@kali: /home/kali/8_jour
Session Actions Edit View Help
root@kali: /home/kali/8_jour x kali@kali: ~ x
Interface tap0 créée. Configurez: sudo ip addr add 10.0.0.1/24 dev tap0 && su
do ip link set tap0 up
SOCKET: Système initialisé.
SOCKET: Création socket ID 1
SOCKET: Socket 1 lié au port 80 (LISTEN)
APP: Serveur Web simulé en écoute sur le port 80 via API Socket.

— Cache ARP Interne —
[0] 10.0.0.1 → 36:b5:15:5a:bf:80
ARP Reply envoyé à 10.0.0.2

— Paquet IP reçu —
Source: 10.0.0.1
Destination: 10.0.0.2
Protocol: 6
Total Length: 60
TCP: Requête de connexion SYN de 10.0.0.1:56126
TCP STATE: LISTEN → SYN_RECEIVED
TCP: SYN-ACK envoyé

— Paquet IP reçu —
Source: 10.0.0.1
Destination: 10.0.0.2
Protocol: 6
Total Length: 40
TCP STATE: SYN_RECEIVED → ESTABLISHED

— Paquet IP reçu —
Source: 10.0.0.1
Destination: 10.0.0.2
Protocol: 6
Total Length: 70
TCP DATA: [30 octets] reçus. Transfert vers Socket API.
TCP: ACK envoyé (SEQ: 9384, ACK attendu: 254318677)

— APPLICATION USER-SPACE —
Reçu 30 octets sur le port 80 :
hello on va run une commande
```

mais si vous aviez fait **echo "un message" | nc 10.0.0.2 81**, "le message dit que aucun socket n'écoute sur le port 81" vous verrez juste ceci

```
root@kali: /home/kali/8_jour
Session Actions Edit View Help
root@kali: /home/kali/8_jour x kali@kali: ~ x kali@kali: ~ x
Total Length: 70
TCP DATA: [30 octets] reçus. Transfert vers Socket API.
TCP: ACK envoyé (SEQ: 9384, ACK attendu: 254318677)

— APPLICATION USER-SPACE —
Reçu 30 octets sur le port 80 :
hello on va run une commande

— Paquet IP reçu —
Source: 10.0.0.1
Destination: 10.0.0.2
Protocol: 6
Total Length: 60
TCP: Requête de connexion SYN de 10.0.0.1:53998
TCP STATE: LISTEN → SYN_RECEIVED
TCP: SYN-ACK envoyé

— Paquet IP reçu —
Source: 10.0.0.1
Destination: 10.0.0.2
Protocol: 6
Total Length: 40
TCP STATE: SYN_RECEIVED → ESTABLISHED

— Paquet IP reçu —
Source: 10.0.0.1
Destination: 10.0.0.2
Protocol: 6
Total Length: 69
TCP DATA: [29 octets] reçus. Transfert vers Socket API.
SOCKET: Données reçues pour le port 81, mais aucun socket ne l'écoute.
TCP: ACK envoyé (SEQ: 887, ACK attendu: 1697379841)

— Cache ARP Interne —
[0] 10.0.0.1 → 36:b5:15:5a:bf:80
ARP Reply envoyé à 10.0.0.2
```

Interprétation :

- Le fait que le message apparaisse sous "APPLICATION USER-SPACE" prouve que votre **abstraction Socket fonctionne**. La stack a fait son boulot de transport et a livré le colis à l'application.
- Le fait que l'application affiche le message envoyée par n'importe qui avec nc prouve la vulnérabilité à l'**Injection de Données** si aucun mécanisme d'authentification n'est en place au niveau applicatif.

2. Focus Sécurité : TCP Data Injection

C'est la capacité d'insérer des données malveillantes dans un flux légitime.

- **Le Scénario :**
 1. Alice est connectée au Serveur (Session établie).
 2. L'Attaquant ("Man in the Middle" ou à l'aveugle) devine/connaît les numéros de séquence (SEQ, ACK).
 3. L'Attaquant envoie un paquet avec le **BON** SEQ mais contenant une commande malveillante (ex: rm -rf /).
- **Le Résultat :** La stack TCP vérifie le SEQ, voit qu'il est valide, et transmet le paquet à l'application. L'application exécute l'ordre.
- **La Contre-mesure :** Le chiffrement (TLS/SSL). Même si l'attaquant injecte des données, il ne peut pas les chiffrer correctement (car il n'a pas la clé de session).

ATTENTION

Le code actuel n'est pas sécurisé car il contient une faille : **L'Injection de Commandes Non Authentiée**. En effet , on n'a pas implémenté de chiffrement.